
Déclaration de COL_MesOutilsOCR

```
// Structure où les informations sont
ST_InfosPatient est une Structure
    EstALD                est un booléen
    TexteBrut              est une chaîne
fin
```

Terminaison de COL_MesOutilsOCR

Procédure globale RecupereVersionCerfa

```
// Résumé : Extrait le numéro de version du document Cerfa
// (ex: "04", "06") à partir du texte brut de l'OCR.
// Syntaxe :
// sVersion = RecupereVersionCerfa (<sTexteOCR> est chaîne)
//
// Paramètres :
//   sTexteOCR (chaîne) : Le texte brut extrait par l'OCR dans le coin supérieur gauche du scan.
// Valeur de retour :
//   chaîne : Le numéro de version du Cerfa
//             sur 2 chiffres (ex: "06"). Renvoie "06" par défaut si aucune version n'est trouvée.
//
// Exemple :
// sVersionDetectee = RecupereVersionCerfa(sTexteZoneHautGauche)
//
```

PROCÉDURE RecupereVersionCerfa(sTexteOCR est une chaîne) : chaîne

sTexteCompact est une chaîne = Majuscule(sTexteOCR)

tabParasites est un tableau de chaînes = [" ", Caract(13), Caract(10), "*", "-", "o", ".", ",", "/", "_"]

```
// 1. Nettoyage MAXIMAL (on vire tout ce qui n'est pas une lettre ou un chiffre)
sTexteCompact = Remplace(sTexteCompact, tabParasites, "")
```

sVersionTrouvee est une chaîne = ""

```
// 2. Recherche Principale Assouplie :
// On autorise l'OCR à confondre le '7' avec un 'T' ou 'Z', et le '4' avec un '0'
sMotif est une chaîne = ".*[1LIF\|]{2}[5S][7TZ][4A0]([0-9]{2}).*"
```

SI RegexVérifie(sTexteCompact, sMotif, sVersionTrouvee) ALORS

 nVersion est un entier = Val(sVersionTrouvee)

 SI 1 <= nVersion <= 20 ALORS

 RENOYER NumériqueVersChaîne(nVersion, "02d")

 FIN

FIN

```
// 3. Secours Assoupli ET Sécurisé :
```

```
// On utilise .*? (non-gourmand) pour s'arrêter aux PREMIERS chiffres après CERFA
```

sMotifSecours est une chaîne = ".*?CERFA.*?([0-9]{2}).*"

SI RegexVérifie(sTexteCompact, sMotifSecours, sVersionTrouvee) ALORS

 nVersion est un entier = Val(sVersionTrouvee)

 SI 1 <= nVersion <= 20 _ET_ nVersion <> 11 ALORS

 RENOYER NumériqueVersChaîne(nVersion, "02d")

 FIN

FIN

RENOYER ""

Procédure globale ALDFinder

```

// Résumé : Analyse les zones des cases à cocher ALD sur le document pour déterminer laquelle est
// remplie. Annule l'opération si l'écart de score est trop faible.

// Syntaxe :
// bResultatALD = ALDFinder (<imgOriginale> est Image, <sVersion> est chaîne)
//
// Paramètres :
//   imgOriginale (Image) : Le scan du document préalablement redressé et rogné.
//   sVersion (chaîne) : La version du Cerfa
//     détectée (ex: "06"). Cette variable peut être modifiée en "ANNULE_ALD" par la procédure si l'analyse
//     est incertaine.

// Valeur de retour :
//   booléen : Renvoie Vrai si la case "ALD exonérante
// " est cochée, Faux si elle n'est pas cochée ou en cas d'annulation.
//
// Exemple :
// stRes.EstALD = ALDFinder(imgScanRedressee, stRes.VersionDetectee)
//

PROCÉDURE ALDFinder(imgOriginale est une Image, sVersion est une chaîne) : booléen

rRatioL est un réel = 0.050 // 5% de la largeur (Zone beaucoup plus large !)
rRatioH est un réel = 0.035 // 3.5% de la hauteur

rX_Exo est un réel; rY_Exo est un réel
rX_Non est un réel; rY_Non est un réel

// Emplacement des cases ALD pour plusieurs versions
selon sVersion
  cas "01", "02"
    rX_Exo = 0.860; rY_Exo = 0.500
    rX_Non = 0.910; rY_Non = 0.500
    rRatioL = 0.030; rRatioH = 0.025
  cas "04", "05"
    rX_Exo = 0.650; rY_Exo = 0.325
    rX_Non = 0.830; rY_Non = 0.325
  cas "06", "07"
    rX_Exo = 0.180; rY_Exo = 0.325
    rX_Non = 0.400; rY_Non = 0.325
  autre cas
    rX_Exo = 0.215; rY_Exo = 0.345
    rX_Non = 0.410; rY_Non = 0.345
fin

// AnalyseRemplissageZone renvoie un % de 0 à 100
rScoreExo est un réel = AnalyseRemplissageZone(imgOriginale, rX_Exo, rY_Exo, rRatioL, rRatioH)
rScoreNon est un réel = AnalyseRemplissageZone(imgOriginale, rX_Non, rY_Non, rRatioL, rRatioH)

SI Abs(rScoreExo - rScoreNon) < 3.0 ALORS
  // L'écart est inférieur à 3% : on affiche le message
  Info("Le scan n'est pas valide, veuillez recommencer.")

  // On modifie la variable sVersion pour prévenir le reste du programme qu'on annule tout !
  sVersion = "ANNULE_ALD"

  RENVOYER Faux
FIN

// =====
// --- DÉBUT DÉBOGAGE VISUEL ---
// On dessine APRÈS l'analyse pour ne pas fausser le comptage
// =====
dDébutDessin(imgOriginale)

// On calcule les pixels exacts à partir de tes pourcentages

```

```

nX_Exo_Abs      est un entier      = rX_Exo * imgOriginale.Largeur
nY_Exo_Abs      est un entier      = rY_Exo * imgOriginale.Hauteur
nX_Non_Abs      est un entier      = rX_Non * imgOriginale.Largeur
nY_Non_Abs      est un entier      = rY_Non * imgOriginale.Hauteur

nLargCase_Abs   est un entier      = rRatioL * imgOriginale.Largeur
nHautCase_Abs   est un entier      = rRatioH * imgOriginale.Hauteur

// On dessine un cadre VERT autour de la case OUI (Exo)
dRectangle(nX_Exo_Abs, nY_Exo_Abs, nX_Exo_Abs + nLargCase_Abs, nY_Exo_Abs + nHautCase_Abs, VertFoncé, 3)
)

// On dessine un cadre ROUGE autour de la case NON
dRectangle(nX_Non_Abs, nY_Non_Abs, nX_Non_Abs + nLargCase_Abs, nY_Non_Abs + nHautCase_Abs, RougeClair, 3)
)
// --- FIN DÉBOGAGE VISUEL ---
// =====

bEstCoche est un booléen = Faux

// Exo est clairement dominant (20% de plus) ET a de la matière (> 1%)
SI (rScoreExo > (rScoreNon * 1.2)) ET (rScoreExo > 1.0) ALORS
    bEstCoche = Vrai
SINON
    // Différence absolue (croix légère)
    // On s'assure que rScoreNon n'est pas lui-même un énorme score
    SI (rScoreExo > rScoreNon + 1.5) ET (rScoreNon < 5.0) ALORS
        bEstCoche = Vrai
    FIN
FIN

// =====
// DÉBUG : PERMET DE VOIR LE POURCENTAGE DE REMPLISSAGE DES CASES ALD
// =====
// sMessageDebug est une chaîne = "Score ALD OUI (Exo) : " + rScoreExo + " %" + RC + ...
// "Score ALD NON : " + rScoreNon + " %" + RC + RC + ...
// "Décision finale : " + (bEstCoche ? "OUI (Exo)" sinon "NON")
//
// Info(sMessageDebug)

RENOYER bEstCoche

```

Procédure globale RedresserEtRogner_PMT

// Résumé : Redresse et rogne le scan original en faisant appel au traitement Java. Gère les erreurs en renvoyant une image vide si le traitement échoue.

```

// Syntaxe :
// imgScanRedressee = RedresserEtRogner_PMT (<IMG_Originale> est Image)
//
// Paramètres :
//   IMG_Originale (Image) : L'image brute du scan à traiter.
// Valeur de retour :
//   Image : L'image finale correctement redressée et rognée. En cas d'échec (scan illisible), renvoie une image vide (Largeur = 0).
//
// Exemple :
// imgScanRedressee = RedresserEtRogner_PMT(imgSource)
//

```

PROCÉDURE RedresserEtRogner_PMT(IMG_Originale est une Image) : Image

bufImage est un Buffer = dSauveImagePNG(IMG_Originale, enMémoire)

```

bufResultat est un Buffer      = _Jv_RedresserEtRogner_ToutEnUn(bufImage)

// Si le Java a planté ou a déclenché sa sécurité (Buffer vide)
SI Taille(bufResultat) = 0 ALORS
    imgVide est une Image // On crée une image totalement vide (Largeur = 0)
    RENVOYER imgVide
FIN

// Sinon tout s'est bien passé
IMG_Proprie est une Image = bufResultat
RENOYER IMG_Proprie

```

Procédure globale _Jv_RedresserEtRogner_ToutEnUn

```

// Résumé : Code natif Java. Redresse et rogne l'image de manière autonome en détectant la bande noire
// principale du document
// Cerfa (méthode des lasers et régression linéaire).
// Syntaxe :
// bufResultat = _Jv_RedresserEtRogner_ToutEnUn (<imageBytes> est Buffer/Tableau d'octets)
//
// Paramètres :
// imageBytes (Buffer / octet[]) : Les données brutes de l'image (en mémoire) à traiter.
// Valeur de retour :
// Buffer
// octet[] : Les données de l'image finale redressée et rognée (compressée en JPEG). Renvoie Null (ou
// un
// Buffer vide dans WINDEV) si le document est illisible, introuvable ou si l'angle est trop fort.
//
// Exemple :
// bufResultat = _Jv_RedresserEtRogner_ToutEnUn(bufImageOriginale)
//

import android.graphics.Bitmap;
import android.graphics.BitmapFactory;
import android.graphics.Matrix;
import android.graphics.Canvas;
import android.graphics.Color;
import android.graphics.Paint;
import android.graphics.RectF;
import java.io.ByteArrayOutputStream;
import java.lang.Math;

public static byte[] _Jv_RedresserEtRogner_ToutEnUn(byte[] imageBytes) {

    if (imageBytes == null || imageBytes.length == 0) return null;

    Bitmap original = null;
    Bitmap rotated = null;
    Bitmap cropped = null;

    try {
        original = BitmapFactory.decodeByteArray(imageBytes, 0, imageBytes.length);
        if (original == null) return null;

        int width = original.getWidth();
        int height = original.getHeight();

        // =====
        // 0. TES VARIABLES DE RECADRAGE SUR MESURE
        // =====
        float margeGauche    = 0.04f;
        float margeDroite    = 0.04f;
        float margeAuDessus  = 0.01f;
        float hauteurCerfa   = 1.25f;
    }

```

```

// =====
// 1. LE MULTI-LASER (Régression Linéaire sur 15 points)
// =====
int nbPointsValides = 0;
long sumX = 0, sumY = 0, sumXY = 0, sumX2 = 0;

// On lance un laser tous les 5% de la largeur de la feuille (de 15% à 85%)
int startX = (int)(width * 0.15);
int endX = (int)(width * 0.85);
int step = (endX - startX) / 15;

for(int x = startX; x <= endX; x += step) {
    int y = trouverBandeNoireCerfa(original, x);
    if(y != -1) {
        nbPointsValides++;
        sumX += x;
        sumY += y;
        sumXY += (long)x * y;
        sumX2 += (long)x * x;
    }
}

// S'il n'a pas trouvé la bande à au moins 5 endroits différents, c'est un mauvais scan
if (nbPointsValides < 5) {
    return null;
}

// =====
// 2. CALCUL DE L'ANGLE MOYEN PARFAIT
// =====
double denominateur = (nbPointsValides * sumX2) - (sumX * sumX);
if (denominateur == 0) return null;

// Calcul de la pente (m) de la droite parfaite
double pente = ((nbPointsValides * sumXY) - (sumX * sumY)) / denominateur;
float angleDeg = (float)Math.toDegrees(Math.atan(pente));

// On calcule aussi la hauteur moyenne (l'ordonnée à l'origine) pour savoir exactement où couper
double b = (sumY - pente * sumX) / nbPointsValides;
int yCentreBandeAvantRotation = (int)(pente * (width / 2.0) + b);

if (Math.abs(angleDeg) > 15.0f) {
    return null; // Angle excessif
}

// =====
// 3. ROTATION
// =====
Matrix matrix = new Matrix();
matrix.setRotate(-angleDeg, width / 2.0f, height / 2.0f);

RectF bounds = new RectF(0, 0, width, height);
matrix.mapRect(bounds);
rotated = Bitmap.createBitmap(Math.round(bounds.width()), Math.round(bounds.height()), Bitmap.
Config.ARGB_8888);
Canvas canvas = new Canvas(rotated);
canvas.drawColor(Color.WHITE);
Paint paint = new Paint();
paint.setFilterBitmap(true);
canvas.drawBitmap(original, matrix, paint);

int rotW = rotated.getWidth();
int rotH = rotated.getHeight();

// =====

```

```

// 4. RECADRAGE SÉCURISÉ (L'Emporte-Pièce)
// =====
float[] centrePoint = new float[] { width / 2.0f, yCentreBandeAvantRotation };
matrix.mapPoints(centrePoint);
int yBandeNoireDroite = (int)centrePoint[1];

int cutLeft   = (int)(rotW * margeGauche);
int cutRight  = (int)(rotW - (rotW * margeDroite));
int cutTop    = (int)(yBandeNoireDroite - (rotW * margeAuDessus));

int largeurUtile = cutRight - cutLeft;
int cutBottom = (int)(yBandeNoireDroite + (largeurUtile * hauteurCerfa));

cutTop = Math.max(0, cutTop);
cutLeft = Math.max(0, cutLeft);
cutRight = Math.min(rotW, cutRight);
cutBottom = Math.min(rotH, cutBottom);

int finalWidth = cutRight - cutLeft;
int finalHeight = cutBottom - cutTop;

if (finalWidth <= 0 || finalHeight <= 0) return null;

// =====
// 5. SORTIE
// =====
cropped = Bitmap.createBitmap(rotated, cutLeft, cutTop, finalWidth, finalHeight);
ByteArrayOutputStream stream = new ByteArrayOutputStream();
cropped.compress(Bitmap.CompressFormat.JPEG, 90, stream);
return stream.toByteArray();

} catch (Exception e) {
    e.printStackTrace();
    return null;
} finally {
    if (original != null && !original.isRecycled()) original.recycle();
    if (rotated != null && !rotated.isRecycled()) rotated.recycle();
    if (cropped != null && !cropped.isRecycled()) cropped.recycle();
}

}

// =====
// LE LASER DE DÉTECTION
// =====
private static int trouverBandeNoireCerfa(Bitmap bmp, int x) {
    int height = bmp.getHeight();
    int width = bmp.getWidth();
    int threshold = 110;

    for (int y = (int)(height * 0.05); y < height * 0.45; y++) {
        int lum = getLum(bmp.getPixel(x, y));

        if (lum < threshold) {
            int verticalBlacks = 0;
            // On vérifie une grosse épaisseur (au moins 12 pixels pleins)
            for (int i = 0; i < 15; i++) {
                if (y + i < height && getLum(bmp.getPixel(x, y + i)) < threshold + 25) {
                    verticalBlacks++;
                }
            }

            if (verticalBlacks >= 12) {
                // Petite sécurité horizontale très locale pour éviter le texte
                int midY = y + 7;
                int blackCount = 0;
                for(int dx = -10; dx <= 10; dx += 2) {

```

```

        if (x+dx >= 0 && x+dx < width && getLum(bmp.getPixel(x+dx, midY)) < threshold + 25)
        {
            blackCount++;
        }
    }
    if (blackCount > 8) {
        return y;
    }
}
}
return -1;
}

private static int getLum(int pixel) {
    return (int)(0.299 * ((pixel >> 16) & 0xFF) + 0.587 * ((pixel >> 8) & 0xFF) + 0.114 * (pixel & 0xFF));
}
}

```

Procédure globale AnalyseRemplissageZone

// Résumé : Découpe une zone ciblée de l'image et analyse son taux de remplissage (présence d'encre/croix) en utilisant un calcul de luminosité dynamique pour s'adapter à l'éclairage.

// Syntaxe :

// rTauxRemplissage = AnalyseRemplissageZone (<imgSource> est Image, <nX_Pct> est réel, <nY_Pct> est réel, <nL_Pct> est réel, <nH_Pct> est réel)

```

//
// Paramètres :
// imgSource (Image) : L'image source (le scan redressé) sur laquelle effectuer la recherche.
//
// nX_Pct (réel) : La position X de départ de la zone, exprimée en pourcentage de la largeur totale (ex: 0.180 pour 18%).
//
// nY_Pct (réel) : La position Y de départ de la zone, exprimée en pourcentage de la hauteur totale.
// nL_Pct (réel) : La largeur de la zone à analyser, en pourcentage.
// nH_Pct (réel) : La hauteur de la zone à analyser, en pourcentage.
// Valeur de retour :
// réel : Le pourcentage (de 0.0 à 100.0) de pixels noirs/foncés détectés dans la zone cible.
//
// Exemple :
// rScoreExo = AnalyseRemplissageZone(imgOriginale, rX_Exo, rY_Exo, rRatioL, rRatioH)
//

```

procédure AnalyseRemplissageZone(imgSource est une Image, nX_Pct est un réel, nY_Pct est un réel, nL_Pct est un réel, nH_Pct est un réel) : réel

```

// 1. Découpe de la zone
nZoneX      est un entier    = Max(0, imgSource.Largeur * nX_Pct)
nZoneY      est un entier    = Max(0, imgSource.Hauteur * nY_Pct)
nZoneL      est un entier    = imgSource.Largeur * nL_Pct
nZoneH      est un entier    = imgSource.Hauteur * nH_Pct

imgZoneTravail est une Image = dDécoupe(imgSource, nZoneX, nZoneY, nZoneL, nZoneH)

// Sécurité : Si découpe invalide
si imgZoneTravail.Largeur = 0 alors renvoyer 0.0

// -----
// DEBUG VISUEL : A DECOMMENTER SI LE RESULTAT EST TOUJOURS BIZARRE
// Cela vous permet de voir ce que l'algo "voit".
// Si vous voyez du noir ici, c'est que les coordonnées (nX_Pct) sont fausses.
// FEN_debug.IMG_Controler = imgZoneTravail
// -----

```

```

// 2. ETALONNAGE : On cherche le "Blanc du papier" (Luminosité Max)
// On parcourt une zone centrale pour éviter les bords noirs éventuels de la découpe
nLuminositeMax    est un entier    = 0
nC                est un entier
nL                est un entier

// On scanne le cœur de l'image (de 20% à 80%) pour trouver le point le plus clair
pour x = (imgZoneTravail.Largeur * 0.2) à (imgZoneTravail.Largeur * 0.8)
    pour y = (imgZoneTravail.Hauteur * 0.2) à (imgZoneTravail.Hauteur * 0.8)
        nC = dPixelCouleur(imgZoneTravail, x, y)
        si nC <> -1 alors
            nL = CouleurLuminosite(nC)
            // On garde la luminosité la plus forte trouvée (c'est le papier)
            si nL > nLuminositeMax alors nLuminositeMax = nL
        fin
    fin
fin

// Si la luminosité max est très faible (<30), c'est que l'image est noire (erreur de zone)
si nLuminositeMax < 30 alors renvoyer 0.0

// 3. DEFINITION DU SEUIL DYNAMIQUE
// On considère comme "Encre" tout pixel qui est nettement plus sombre que le papier.
// On fixe le seuil à 70% de la luminosité du papier.
// Ex: Papier à 200 -> Seuil à 140. Tout ce qui est < 140 est compté.
// Ex: Papier sombre (ombre) à 100 -> Seuil à 70.
nSeuilDetection    est un entier    = nLuminositeMax * 0.70

// 4. COMPTAGE (Zone centrale uniquement)
nPixelsNoirs    est un entier    = 0
nTotalPixels    est un entier    = 0

// On réduit légèrement la zone de comptage (marge de 10%) pour éviter le cadre de la case
nDebutX        est un entier    = imgZoneTravail.Largeur * 0.10
nFinX          est un entier    = imgZoneTravail.Largeur * 0.90
nDebutY        est un entier    = imgZoneTravail.Hauteur * 0.10
nFinY          est un entier    = imgZoneTravail.Hauteur * 0.90

pour x = nDebutX à nFinX
    pour y = nDebutY à nFinY
        nC = dPixelCouleur(imgZoneTravail, x, y)
        si nC <> -1 alors
            nL = CouleurLuminosite(nC)
            // Si le pixel est plus sombre que notre seuil dynamique
            si nL < nSeuilDetection alors
                nPixelsNoirs++
            fin
            nTotalPixels++
        fin
    fin
fin

si nTotalPixels = 0 alors renvoyer 0.0

// 5. Renvoie le pourcentage
renvoyer (nPixelsNoirs / nTotalPixels) * 100

```